ACCEPTED FROM OPEN CALL

Re-Architecting NFV Ecosystem with Microservices: State of the Art and Research Challenges

Shihabur Rahman Chowdhury, Mohammad A. Salahuddin, Noura Limam, and Raouf Boutaba

Abstract

Network Function Virtualization (NFV), considered a key enabler of network "softwarization", promises to reduce capital and operational expenditures for network operators by moving packet processing from purpose-built hardware to software running on commodity servers. However, the state-of-the-art in NFV is merely replacing monolithic hardware with monolithic VNFs, the software that realizes different network functions (e.g., firewalls, WAN optimizers, and so on). Although this is a first step toward deploying NFV, common functionality is repeatedly implemented in monolithic VNFs. Repeated execution of such redundant functionality introduces processing overhead when VNFs are chained to realize Service Function Chains and leads to sub-optimal usage of infrastructure resources. This stresses the need for re-architecting the NFV ecosystem, from VNFs to their orchestration, through modular VNF design and flexible service composition. In that perspective, we make the case for using the microservice software architecture, proven to be effective for building large-scale cloud applications from reusable and independently deployable components, to re-architect the NFV ecosystem. We also discuss the state-of-the-art in realizing modular VNFs from both industry and academia. Finally, we outline a set of research challenges for microservice-based NFV platforms.

Introduction

Network operators ubiquitously deploy hardware *middleboxes* (e.g., firewalls, WAN optimizers, and so on) to realize different security and performance goals and to offer value-added services (e.g., parental control, video-on-demand, and so on). The number of middleboxes in a network is typically in the same order as the number of switches and routers [1]. However, the middlebox ecosystem is in many ways similar to the mainframe industry in the early '80s, that is, vendor specific, purpose-built and vertically integrated hardware, software and control with limited to no programmability. Such a closed and inflexible ecosystem forces network operators to resort to manual and error-prone methods for deploying and configuring network services. Deployment and configuration scenarios become further complicated when network operators need to steer traffic through an ordered sequence of middleboxes, that is, a Service Function Chain (SFC) (defined in IETF RFC7498). The inflexibilities in the middlebox ecosystem lead to increased operational complexity and expenditure, delaying time to market for new services.

More recently, the networking industry is going through a transformation known as *network softwarization*. Network softwarization refers to the general practice of decoupling network processing and control software from specialized hardware. This enables network operators to replace vendor-specific, purpose-built networking equipment with commodity hardware and leverages open APIs to control and provision the networking infrastructure in a programmatic way. A key enabler for network softwarization is Network Functions Virtualization (NFV) [2]. NFV proposes to virtualize Network Functions (NFs), that is, functionality realized by the hardware middleboxes, by decoupling the traffic processing software from purpose-built hardware. In this way, NFV enables the operators to replace vendor specific, expensive and vertically integrated hardware middleboxes with Virtual Network Functions (VNFs) running on commodity off-the-shelf servers. This increases flexibility and re-usability of the same hardware for multiple NFs. Hence, NFV brings economies of scale and leverages the advances in application orchestration for on-demand deployment and scaling of SFCs.

NFV is envisioned to increase the agility of the communication infrastructure to support future applications, for example, IoT, smart grid, smart cities, connected drones, and so on. However, state-of-the-art NFV platforms (e.g., OPNFV (https://www.opnfv.org/), OpenMANO (https://osm.etsi.org/)) are merely replacing monolithic hardware NFs with their monolithic software counterparts. Definitely, this is the first logical step toward network softwarization transformation. However, monolithic VNFs in SFCs can lead to sub-optimal resource usage and hinder infrastructure agility.

A fundamental problem with monolithic VNFs is that a large number of common functionalities (e.g., packet header parsing, payload inspection, packet classification, and so on) are repeated across different VNFs developed by various third-party providers. This has several negative consequences, including:

- Redundant development of functionalities in different VNFs.

Digital Object Identifier: 10.1109/MNET.2019.1800082

The authors are with the University of Waterloo.

0890-8044/19/$25.00 © 2019 IEEE

- Unnecessary processing overhead for executing these redundant functionalities when VNFs are chained to form SFCs (shown to exceed 25 percent for some SFCs [3]).
- Coarse-grain resource allocation and scaling imposed by the monolithic nature of VNFs.

These limitations stress the need to rethink how VNFs can be developed and orchestrated for agile service creation and scaling [4].

In this article, we make the case for re-architecting the current monolithic VNFs and their orchestration to eliminate inherent redundancies and exploit their commonalities through modular VNF design and flexible service composition. To this end, we envision the adoption of the *microservice software architecture* [5] for building, deploying and managing VNFs. The microservice software architecture decomposes monolithic software into independently deployable components with well-defined interfaces, called *microservices*. It has been proven effective for building large-scale cloud applications. In the remainder of this article, we start by providing background on the microservice software architecture. Then we make our case for using microservices to re-architect VNFs through an illustrative example. This is followed by a discussion of recent efforts from industry and academia to develop modular VNFs and their supporting runtime systems. Finally, we outline several research challenges that we have identified for realizing microservice-based VNFs.

Microservice Software Architecture

The National Institute of Standards and Technology (NIST) defines microservices as follows: "*A microservice is a basic element that results from the architectural decomposition of an application's components into loosely coupled patterns consisting of self-contained services that communicate with each other using a standard communications protocol and a set of well-defined APIs, independent of any vendor, product or technology*" [5]. From this definition, we draw the following key features of microservices.

Self-Contained: A microservice typically performs a single task and does not depend on other microservices to perform its task.

Loosely Coupled: Generally, a microservice does not require tight synchronization with other microservices to operate.

Well Defined Communication Interfaces: In a microservice architecture, an application completes a series of tasks by different microservices. For this purpose, microservices must exhibit well-defined APIs that an orchestrator and other microservices can use for communication. A popular choice for such interface is the RESTful API. However, message queues and protocol buffers are competing alternatives.

A major advantage of using microservices is the ability to allocate resources at a finer granularity. For example, consider a monolithic application in Fig. 1a, composed of tightly coupled components, *C*1, *C*2 and *C*3. A surge in demand requires increased processing in components *C*1 and *C*2 of the application. In this monolithic application, all components need to be scaled because of the tight coupling, resulting in idle resources allocated to *C*3. In contrast, consider Fig. 1b, where the same application is re-architected using microservices *C*1′, *C*2′ and *C*3′. Now the scaling of the application is more efficient in terms of resource allocation. Additionally, the self-contained and loosely coupled nature of microservices facilitate independent development and maintenance.

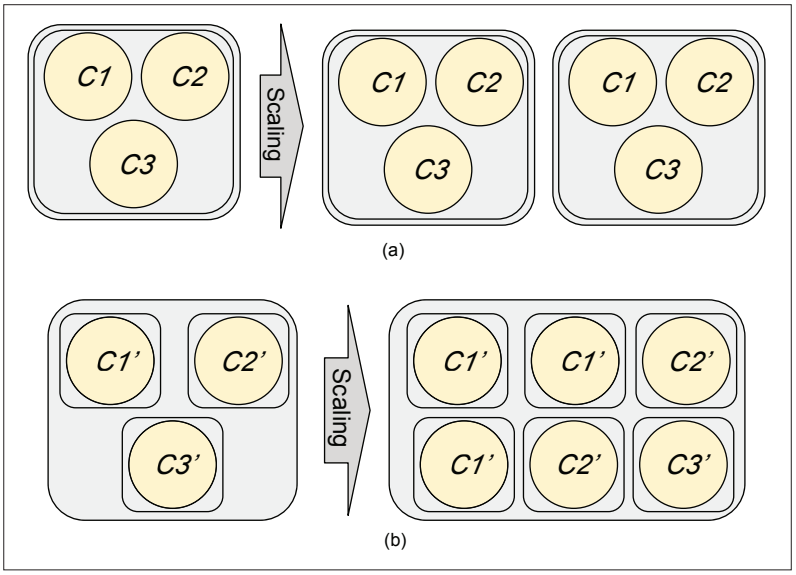

FIGURE 1. Monolithic vs. microservice-based application: a) a monolithic application with tightly coupled components, inefficient scaling and coarse grain resource allocation; b) a microservice-based application with independently scalable microservices, facilitating fine grain resource allocation.

The Case for Adopting Microservices for NFV

In this section, we motivate the adoption of microservice software design principles for re-architecting VNFs. We use the example SFC in Fig. 2a, which is described in an Internet draft for SFC use cases in data center networks (https://tools.ietf.org/html/draft-ietf-sfc-dc-use-cases-06), to illustrate the common functionalities between different VNFs. The details of each function in Fig. 2a can be found in the Internet draft. In the following, we briefly describe the specific functionality for our subsequent discussion.

WAN Optimizer: Compresses/decompresses HTTP payload (e.g., text, image) on egress/ingress traffic to optimize bandwidth usage on the WAN links that connect a data center to another peer data center.

Edge Firewall: Performs access control based on layer 2—4 headers, for example, IP subnet, TCP port, and so on.

Monitoring Function: Performs the following operations: computes the packet size distribution from network traffic; and counts the number of packets destined to an IP subnet inside the data center network.

Application Firewall: Blocks incoming HTTP requests that have an SQL injection attack embedded in the URL.

Load Balancer: Distributes traffic to the backend servers based on the hash of layer 3–4 headers.

We perform a functional decomposition of each of the aforementioned VNFs and present it in Fig. 2b. Each block in Fig. 2b represents an operation performed on a packet while the arrows represent the flow of operations. We observe the following from the decomposition.

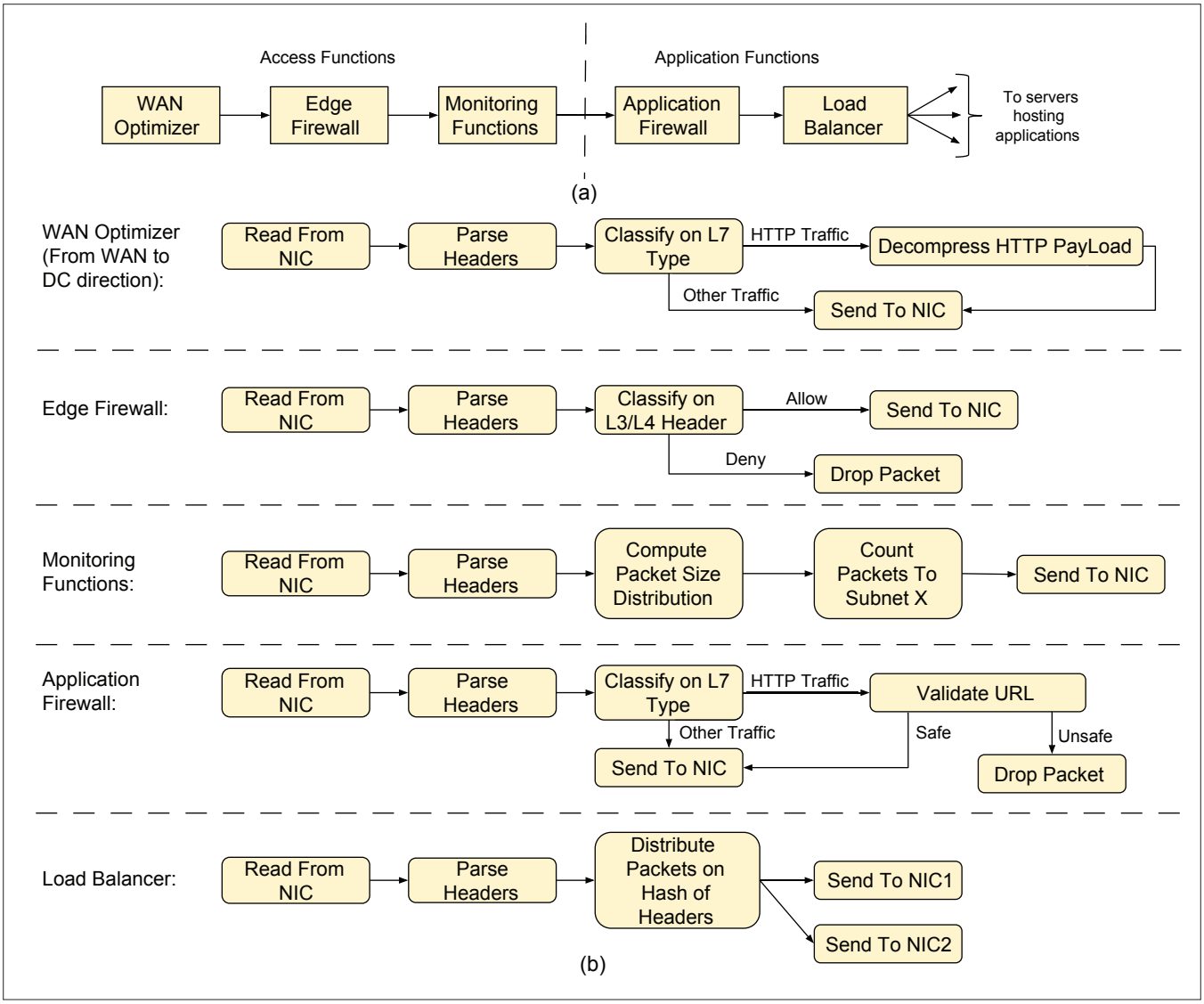

FIGURE 2. Illustrative example to motivate microservice-based VNFs: a) example SFC for data center networks; b) functional decomposition of the VNFs in Fig. 2a.

Overlapped Functionality: Many of the functionalities are common across VNFs. In some cases, the functionality is the same but the parameters are different (e.g., writing a packet but to different network interface cards (NICs)), while in other cases both the functionality and the configuration are exactly the same (e.g., HTTP packet classification performed by both the WAN optimizer and application firewall).

Wasted CPU Cycles: Since a functionality is embedded within a monolithic VNF, its execution result is not easily reusable across VNFs (e.g., classification of a packet as HTTP by the WAN optimizer is not usable by the application firewall). Therefore, CPU cycles are wasted due to the repeated execution of the same functionality when a packet goes through the VNFs in an SFC. This is also experimentally validated by prior research, reporting more than 25 percent CPU cycles are wasted for such redundant processing for certain SFCs [3].

Inflexible Scaling: It is difficult to scale resources for individual functionalities since they are embedded into monolithic VNFs. If a functional block in Fig. 2b becomes a bottleneck, the whole VNF needs to be scaled up/out (e.g., there is no easy way to allocate additional CPU resources for performing URL validation in an application firewall in Fig. 2b without scaling up/out the whole VNF instance). This requires more resources compared to scaling up/out resources for a single functionality.

These observations motivate the adoption of microservices for re-architecting the VNFs. Applying the microservice software design principle to VNFs will result in a set of independently deployable packet processing microservices with well-defined interfaces corresponding to the functional blocks in Fig. 2b. In what follows, we use the term μNF to refer to these packet processing microservices. An SFC will then be realized by orchestrating a processing graph composed of μNFs while removing repeated application of the same packet processing functionality. As a result, it will be possible to recover the CPU cycles spent in redundant processing while performing finer grain resource allocation.

μNFs are to some extent analogous to the VNF components (VNFCs) described in ETSI's VNF

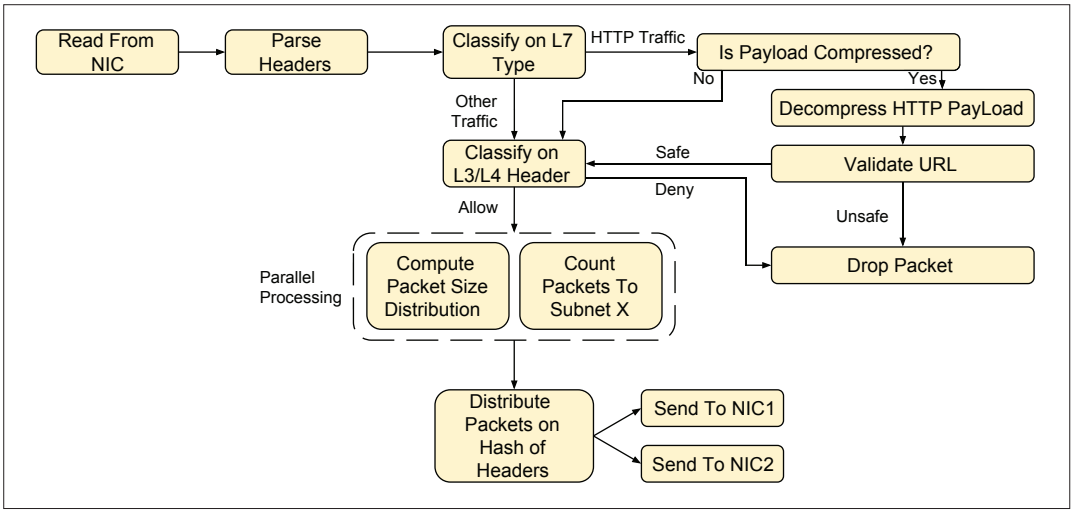

FIGURE 3. Microservice-based realization of the SFC in Fig. 2a.

architectural description [6]. According to [6], a VNF consists of one or more VNFCs, where each VNFC has a 1:1 correspondence with a virtualization container (e.g., VM, OS container, and so on). However, a major difference between a µNF and a VNFC is that µNFs are loosely coupled components that can be used across VNF boundaries, whereas VNFCs are not. This property also makes SFC orchestration different in the case of µNFs since there are more optimization opportunities for µNFs compared to VNFCs.

In Fig. 3, we present an optimized realization of the SFC from Fig. 2a using µNFs to illustrate some of the optimization opportunities. Assuming that mechanisms to propagate output of one µNF to the others exist, we can consolidate packet processing functions such as packet I/O, header parsing, HTTP packet classification, and so on. Furthermore, some µNFs can be executed in parallel, such as the counting functionality performed by the monitoring functions that do not modify packets. These optimizations are only possible due to the inherent loosely coupled characteristic of µNFs.

State-of-the-Art

The history of modular packet processing using software dates back to the late '90s with the introduction of the *Click* [7] modular router. However, Click is not built based on the principles of microservices. It compiles a monolithic packet processing software from a set of reusable packet processing elements. The concept of modularization in Click motivated later efforts to address the shortcomings of monolithic VNFs. Although these endeavors did not always explicitly label their approach as using microservices, they follow the principles of a microservice architecture. In this section, we briefly discuss the efforts from both academia and industry that address the shortcomings of monolithic VNFs through a modular design.

Academic Efforts

CoMb [3] is one of the early endeavors that partially addressed the shortcomings of monolithic VNFs. CoMb proposed to share some low-level common functionality between VNFs (e.g., TCP session reconstruction). However, apart from a set of low-level common functionalities, the VNFs in CoMb were monolithic. Some recent efforts in the literature have proposed to replace monolithic VNFs with reusable data path components that can be controlled and configured from a centralized control plane, while others have developed runtime systems to support VNFs composed from these data plane processing functions. In the following, we discuss the state-of-the-art academic efforts and categorize them based on architectures for µNF-based VNFs and runtime systems to support µNF-based VNFs, respectively.

µNFs-based VNF Architectures: OpenBox [8] and Microboxes [9] are two recent endeavors that address the shortcomings of monolithic VNFs. Both propose an architecture for composing VNFs from lightweight and reusable packet processing components. Their proposed architectures have similarities and can be generalized as depicted in Fig. 4. They realize VNFs and SFCs using a composition of lightweight packet processing modules (OpenBox Service Instance (OBSI) in OpenBox and µStack in Microboxes) running on commodity servers or possibly on specialized hardware. OBSIs and µStacks are analogous to µNFs discussed previously. An application written in a high-level language (e.g., Python) determines the composition of OBSIs or µStacks (cf. Middlebox Apps in Fig. 4). Necessary abstractions for writing applications are provided by a middlebox runtime system (*OpenBox control plane* or *Microboxes controller*). The runtime system is also responsible for orchestration tasks such as placing OBSIs or µStacks and steering traffic between them.

A fundamental difference between OBSIs and µStacks is the way they are executed in a packet processing pipeline. OBSIs are executed sequentially and interact with each other using a packet-centric communication model, that is, a batch of packets is sequentially processed by the OBSIs in a packet processing pipeline until the batch exits the system. In contrast, µStacks are executed asynchronously and in parallel, and communicate with each other using an event based publish/subscribe model. A µStack publishes TCP processing events (e.g., connection initiation or termination) and the associated meta-data (e.g.,

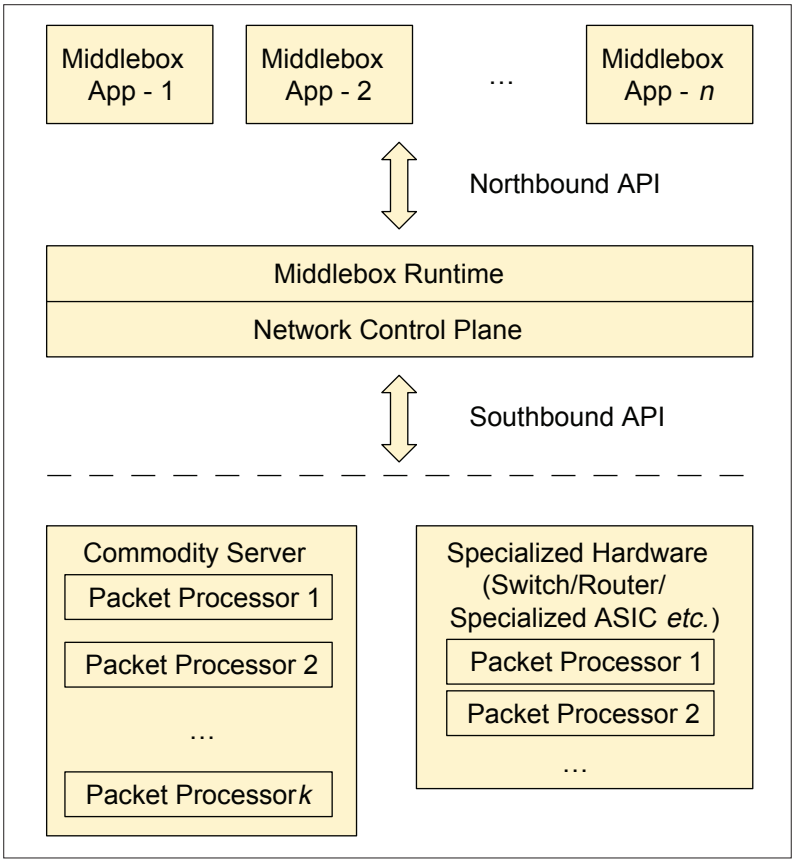

FIGURE 4. Generalized architecture of OpenBox [8] and Microboxes [9].

pointer to SYN or FIN packets) to other µStacks that have subscribed to these events. The subscribed µStacks are then executed, possibly in parallel on the packets from the same flow. The sequential execution of OBSIs in OpenBox is simpler and easier to reason when compared to the asynchronous and parallel execution of µStacks in Microboxes. However, the latter reduces packet processing latency and better utilizes the CPUs of the underlying machines.

Runtime Systems for µNF-based VNFs and SFCs: Flurries [10] and NetBricks [11] address issues related to the runtime system for supporting VNFs composed of components built around the concept of µNFs. However, they focus on different aspects of the runtime system. Flurries proposes an efficient packet exchange mechanism between µNFs running on Docker containers, whereas NetBricks proposes isolation mechanisms between multiple instances of µNF-like components without using VMs or OS containers.

Flurries assumes the existence of thousands of lightweight NFs (similar to µNFs) running in Docker containers, each performing a small packet processing task while processing packets from a single flow. Under these assumptions, Flurries proposes the design of a data plane for the NFs. The proposed data plane provides zero-copy packet exchange mechanism between the NF containers deployed on the same machine. It performs flow classification and coordinates with a scheduler to assign new flows to new NFs. A zero-copy packet exchange mechanism is provided by using the shared memory primitives from the Intel DPDK library. The NFs running in containers use a hybrid of interrupt and polling techniques to maximize throughput while not always using 100 percent of the CPU. However, the concept of ownership transfer as provided by virtual switches (e.g., Open vSwitch) is not implemented in Flurries. Ownership transfer ensures that an NF that has finished processing a packet should not be able to use the memory handlers to modify the same packet in the future.

The authors of NetBricks argue through an experimental study that current virtualization technologies (e.g., VMs, OS containers) cannot meet the performance requirement of NFV. According to their study, packet processing performance for 64 byte packets drops by 3× and 7× for containers and VMs, respectively, when compared to bare-metal performance. NetBricks addresses these limitations by designing a process-based runtime system, which provides the same memory isolation as containers and VMs without incurring the performance overhead. It also provides packet processing abstractions to compose VNFs. These abstractions are similar to the concept of µNFs, that is, they also separate the packet processing and the pipeline composition.

However, NetBricks compiles everything into one NetBricks process and runs each SFC in a separate thread inside that process. Within one SFC, that is, within one thread in NetBricks, a NF passes its processed packets to the next NF by invoking the next NF's appropriate method, which in turn ensures the transfer of packet ownership. Furthermore, NetBricks relies on Rust, a memory-safe programming language that prevents unsafe memory operations performed by multiple threads. Although NetBricks does not strictly adhere to the concept of µNFs, the proposed runtime system can be a potential choice for deploying µNFs.

Flurries and NetBricks address specific issues regarding the runtime system, that is, packet exchange mechanism and memory isolation. However, they do not address issues such as µNFs placement, fault-tolerance, state management, and so on. They rely on external systems to provide such functionalities.

Comparison between Academic Approaches

Table 1 presents a comparative study of the academic efforts based on the following features.

Abstraction for Processing: This refers to the level of abstraction exposed by the underlying runtime system for developing µNFs (e.g., packet, flow, event). Also, a µNF can have "one-to-one" or "one-to-many" relationship with the processing granularity, for example, one µNF per flow (one-to-one relationship between µNF and flow), one µNF for a set of flows (one-to-many relationship between µNF and flow).

Placement and Resource Allocation: This relates to the process of determining the placement and the number of µNFs, and allocating sufficient resources to meet the desired QoS requirements of the SFC. We classify the proposals based on whether they integrate such mechanisms or rely on an external system.

Fault-Tolerance: This pertains to the necessary steps to ensure high availability of the SFCs.

Inter µNF Communication: The mechanism used by µNFs to exchange packets or events

State-of-the-art	Feature					
	Abstraction for processing	Placement and resource allocation	Fault tolerance	Inter μNF communication	Memory isolation mechanism	Ownership transfer
Microboxes [9]	Event (one-to-many)	N.A.	N.A.	Zero-copy shared memory	N.A.	Not transferred
OpenBox [8]	Flow (one-to-many)	Heuristic	N.A.	N.A.	VM	N.A.
Flurries [10]	Flow (one-to-one)	External	N.A.	Zero-copy shared memory	Docker container	Not transferred
NetBricks [11]	Packet, bytestream (one-to-many)	External	External	Method invocation inside same address space	Threads inside processes built using Rust's memory safe operations	Inter-NF procedure call marks packets to ensure ownership transfer
N.A. = Not addressed						

TABLE 1. Comparison of the academic state-of-the-art.

between them. This can be based on shared memory between processes, through a virtual switch, through method invocation, and so on

Memory Isolation Mechanism: This refers to the mechanisms adopted to provide memory isolation between μNFs. Possible options include using VMs or OS containers, resorting to memory safe programming language and runtime system (e.g., Rust and LLVM).

Ownership Transfer: This ensures that once a μNF has sent a packet to another μNF, it cannot modify the packet using the previously used packet handlers.

The above set of features, though not exhaustive, constitutes the collective feature set inspired from the state-of-the-art. The comparison based on these features provides an insight into the lack of maturity of this area of research and highlights the opportunities for further research. For example, we can see that the ownership transfer between multiple OS containers without copying packets is an open issue.

Industry Efforts

There has been some effort from the industry to re-architect VNFs using microservices. The Central Office Re-architected as Datacenter (CORD) project, led by Open Networking Lab (ON.Lab), focuses on transforming operator central offices by replacing the purpose-built hardware with commodity hardware. CORD also virtualizes a number of middleboxes such as Optical Line Terminator, Broadband Network Gateway, and so on to their virtual counterparts. A design approach taken during the process was to decouple the control and processing functions of these middleboxes and run them in separate VMs. As a result, scaling out data processing functionality does not incur the overhead of scaling redundant control plane functionalities. This decoupling can be considered as a step toward microservice decomposition for NFs.

Another industry effort to leverage the benefits of microservices in telecommunication networks is from Metaswitch Networks. Clearwater IMS (http://www.projectclearwater.org/technical/clearwater-architecture/) is their recent IP Multimedia Subsystem (IMS), which has been developed using the microservices design principle. Clearwater IMS can be deployed as a collection of independent software components, where each component can be horizontally and independently scaled. Clearwater stores the sate of each component in persistent storage, thus eliminating the complexity of state management while scaling the components. However, the components themselves are large monolithic software components including a substantial number of tightly coupled functionalities.

A fundamental difference observed between the industry and academic efforts is the granularity of microservice decomposition. Industry efforts have favored coarse-grain decomposition, compared to the fine-grain decomposition at the level of packet operations, promoted by academic projects.

Research Challenges

Designing, deploying and managing microservice-based software comes with its own set of challenges. Microservice-based NFV platforms inherit these challenges and add new dimensions due to inherent differences between VNFs and cloud applications. These differences include several orders of magnitude tighter latency overhead budget for VNFs compared to cloud applications, and higher reliability requirements (five nines) for network operators than cloud service providers (four or four and a half nines). In this section, we discuss the research challenges confronting the realization of microservice-based NFV ecosystems. Interested readers are directed to [12] and [13] for issues specific to microservices and NFV, respectively.

Microservices Decomposition of VNFs

The first challenge in realizing any microservice-based software is to identify the set of microservices. This is difficult because it requires domain-specific knowledge. In the case of NFV, we need to identify and develop a set of sufficiently generic μNFs that are able to realize a wide range of VNFs. One way to approach VNF decomposition is to leverage domain expertise (e.g., by consulting with VNF developers) or to study existing open-source VNFs and identify smaller functional units. A μNF may perform one or more low-level protocol processing tasks corresponding to each layer in the TCP/IP stack. For example, classify packets on layer-2/3 header, re-construct a TCP stream, and so on, or application-layer specific processing such as compressing or decompressing text/image in an HTTP payload. Indeed, a key research issue here is to

From an architectural perspective, microservices are an attractive choice for re-designing network functions. However, from a practical standpoint such redesign can also add overhead to packet processing. A systematic study is required to understand the trade-offs between the observed overhead and the achieved flexibility.

determine the granularity of such μNF operations, that is, single packet processing operation vs. a collection of operations performing a single functionality. On one hand, recent developments in modular VNF design propose low-level packet processing modules as the VNF building blocks ([8, 9]). On the other hand, state-of-the-art commercial VNFs (e.g., Clearwater IMS) are being constructed from coarse-grain microservices. There are advantages and disadvantages of both approaches. A fine-grain decomposition facilitates better re-usability at the cost of increased communication overhead and higher packet processing latencies. In turn, coarse-grain decomposition reduces re-usability but has the potential to reduce the aforementioned overhead. Therefore, it will be very useful to theoretically analyze and empirically evaluate this trade-off to determine the granularity that is most beneficial for VNF decomposition.

Communication Primitives for VNF Composition

Microservice software architecture mandates that each microservice has a well-defined interface for communicating with other microservices. Traditionally, cloud applications built using microservices have been using RESTful APIs or a message broker for communication, which will incur significant overhead when used for VNF composition. In the case of composing VNFs from μNFs, the communication interface should have the following properties:

- Minimal impact (order of few microseconds) on the overall packet processing latency.
- Ensure packet ownership transfer between μNFs.
- Allow microservices to pass the result of their processing to other microservices to eliminate redundant processing.
- Enable μNF–μNF communication across physical machine boundaries.

Two design extremes of such a communication interface are to perform a packet copy between microservices and to share packets using a shared memory, respectively. The first approach has significant performance implications albeit providing isolation, whereas the latter will incur low latency at the cost of isolation.

Resource Allocation for QoS-aware VNF Composition

The ability to use μNFs beyond VNF boundaries creates opportunities for optimizing resources allocated to SFCs. Recall from above that there are possibilities of consolidating multiple μNFs and also parallelizing the execution of a subset of μNFs in an SFC. A systematic approach is required to develop algorithms for resource allocation for μNFs that take the aforementioned possibilities into account while meeting QoS requirements (e.g., throughput). Other possibilities to consider include reordering the μNFs without changing the SFC semantics to improve existing placements or make new ones feasible, which were not possible with monolithic VNFs. Also, algorithms for dynamically scaling microservices to meet changes in traffic demand also need attention from the research community.

Microservice Performance Profiling

A key to efficient resource allocation and VNF management is to have accurate resource usage and performance profiles of the μNFs. The performance profile of μNFs will contain resources required (e.g., CPU share) to achieve certain QoS parameters (e.g., packets processed per second) under a given workload. Such profiles help the orchestration system to optimize μNF placement and the operators to reason about the perceived performance. This non-trivial task requires developing workloads that represent realistic scenarios and instrumentation mechanisms to measure different performance parameters under various loads.

Overhead vs. Flexibility

From an architectural perspective, microservices are an attractive choice for re-designing network functions. However, from a practical standpoint such redesign can also add overhead to packet processing (e.g., increased processing latency due to long paths in a μNF processing graph). A systematic study is required to understand the trade-offs between the observed overhead and the achieved flexibility. Such a study will help to identify the use cases where microservice-based NFs are more beneficial.

Language for VNF and SFC Composition

A fundamental requirement in a microservice-based NFV platform is a language that can capture the hierarchical nature of SFC composition, that is, how VNFs are composed using μNFs (a VNF template) and subsequently how SFCs will be composed from VNFs (SFC specification). When a network operator defines an SFC by describing the VNFs and their interconnections, the first step for the orchestrator is to translate the SFC or a set of SFCs to a μNF processing graph by leveraging a set of predefined VNF templates. This translation acts as a basis for further optimizing the μNF graph for resource allocation. Available orchestration languages such as TOSCA (https://docs.oasis-open.org/tosca/tosca-nfv/v1.0/tosca-nfv-v1.0.html) and HOT (http://docs.openstack.org/developer/heat/template_guide/hot_guide.html) can be evaluated for possible adoption or new languages need to be developed.

Fault Tolerance and Recovery

In general, fault tolerance has been a key concern for microservice-based software. Re-usability, a key feature of microservices, is also responsible for causing cascading failures. However, the situation becomes even more critical for telecommunication network operators who typically require higher reliability (five nines or more) compared to cloud service providers (four nines). A major challenge in realizing microservice-based NFV platforms is to ensure strong consistency of μNF states while recovering from failures within an order of milliseconds. And all this needs to

be achieved with an overhead budget of a few microseconds per packet.

Stateless vs. Stateful Microservice Design

A design choice in building microservice-based VNFs is to determine the placement of states (e.g., address translation table). One approach is to keep the states with the microservices for easier maintenance. However, this makes scaling difficult due to state synchronization. Another approach is to maintain the VNF-specific state external to the microservices and have a separate logic for handling it (e.g., similar to [14]). This decouples the microservices from stateful network processing and facilitates easier scaling. However, this can add to the packet processing latency due to remote state access. Therefore, the challenge lies in striking a balance between ease of scaling and maintenance overhead in such a platform.

Virtualization Platforms

Virtualization provides isolation between coexisting applications. However, the level of virtualization has an impact on the level of isolation and the performance overhead. A popular choice for deploying microservices in the cloud is containers (e.g., Docker). However, studies show severe performance degradation in packet processing throughput even when using containers [11]. Ideally, the virtualization platform should have minimal performance impact compared to bare-metal and should be able to spawn μNFs within milliseconds to support on-demand service provisioning. This mandates the need for new virtualization platforms to support μNFs. One research direction is to explore unikernel operating systems [15] to create a lightweight virtualization platform for μNFs.

Underlying Computer Architectures

Appropriate computer architectures to support μNF-based VNFs is an important consideration. State-of-the-art multi-core processors have tens of processing cores, reaching up to hundreds with multiple CPU sockets. However, inter-core communication and synchronization is considered expensive, let alone its inter-socket counterpart. A possible area of exploration is to use processors with hundreds of small and power-efficient cores to allow many microservices to be co-located on a single machine. In this regard, rack-scale computing architectures, that is, servers with a large number of tightly integrated systems-on-a-chip, is a possible candidate for exploration.

Monitoring

Monitoring is an integral part of any network management system. A key challenge in microservice-based NFV platforms will be to derive end-to-end metrics (e.g., end-to-end latency) from μNF-specific monitoring data (e.g., per packet latency). The challenges come from the fact that packets can take different paths in the μNF processing graph. Therefore, tracing the end-to-end packet life-cycle will require understanding the interaction between microservices. This will require developing the necessary infrastructure to ingest μNF–μNF interaction and μNF-specific metrics, and to analyze the data to obtain end-to-end performance metrics. Another design consideration is to equip the microservices with necessary monitoring hooks (e.g., counters, filters and so on) in the first place to facilitate the derivation of end-to-end performance metrics.

Debugging and verification are essential tools for both the developers and the network operators. This has been a well-investigated research topic for different kinds of networks. An additional challenge in μNF-based deployment will be the scale of the problem.

Debugging and Verification

Debugging and verification are essential tools for both the developers and the network operators. This has been a well-investigated research topic for different kinds of networks. An additional challenge in μNF-based deployment will be the scale of the problem. A μNFs-based deployment will increase the scale of NFV deployment by at least an order of magnitude. In addition, the debugging and verification tools must be able to capture the interaction between different μNFs. Capturing the interaction between μNFs, analyzing the interaction to capture the semantics of SFCs, verification of the semantics against network policies and identifying the root cause of errors are some interesting research problems to explore.

Co-Existence with Traditional NFV Ecosystem

The transformation from monolithic VNFs to μNF-based ones will not happen overnight. Rather, we envision the two ecosystems to co-exist for some time. This poses challenges in designing the orchestrator, for instance. In a hybrid deployment, the orchestrator should be able to communicate with both monolithic VNFs and μNFs. There is also room for alternative deployment models where each ecosystem has its own local orchestrator, managed by a global orchestrator.

Conclusion

In this article, we motivate the adoption of the microservice architecture for re-architecting the NFV ecosystem. The ultimate goal is to speed up innovation in NFV by:

- Allowing independent development of small pieces of the ecosystem.
- Improving the utilization of underlying infrastructure, thus positively impacting operational costs.
- Contributing to increasing the infrastructure agility for supporting the next generation of applications.

We also surveyed the state-of-the-art from both industry and academia. Our survey indicates that research in this field is still in its infancy and provides great opportunities for future contributions. The research challenges presented in this article can constitute a starting point for interested readers in this exciting area.

Acknowledgment

This work was supported in part by the NSERC CREATE for Network Softwarization program.

References

[1] J. Sherry *et al.*, "Making Middleboxes Someone Else's Problem: Network Processing as a Cloud Service," *ACM SIGCOMM Computer Comm. Rev.*, vol. 42, no. 4, 2012, pp. 13–24.

[2] "Network Functions Virtualisation Introductory White Paper," white paper, Oct. 2012, accessed: Feb 05, 2017; available: https://portal.etsi.org/nfv/nfv white paper.pdf
[3] V. Sekar *et al.*, "Design and Implementation of a Consolidated Middlebox Architecture," *Proc. USENIX NSDI '12. USENIX Association*, 2012, pp. 24–24.
[4] R. Roseboro, "Cloud-Native NFV Architecture for Agile Service Creation & Scaling," white paper, Jan. 2016.
[5] "NIST Definition of Microservices, Application Containers and System Virtual Machines," draft, Feb. 2016, accessed: Feb. 05, 2017; available: http://csrc.nist.gov/publications/drafts/800- 180/sp800-180 draft.pdf.
[6] "Network Functions Virtualisation (NFV); Virtual Network Functions Architecture," white paper, Dec. 2014, accessed: Feb 05, 2017; available: http://www.etsi.org/deliver/etsi gs/NFVSWA/ 001 099/001/01.01.01 60/gsNFV-SWA001v010101p.pdf.
[7] R. Morris *et al.*, "The Click Modular Router," *Proc. ACM SOSP '99*, ACM, 1999, pp. 217–31.
[8] A. Bremler-Barr, Y. Harchol, and D. Hay, "Openbox: A Software-Defined Framework for Developing, Deploying, and Managing Network Functions," *Proc. ACM SIGCOMM '16*, ACM, 2016, pp. 511–24.
[9] G. Liu *et al.*, "Microboxes: High Performance NFV with Customizable, Asynchronous TCP Stacks and Dynamic Subscriptions," *Proc. ACM SIGCOMM 2018*, ACM, 2018, pp. 504–17.
[10] W. Zhang *et al.*, "Flurries: Countless Fine-Grained NFS for Flexible Per-Flow Customization," *Proc. ACM CoNEXT '16*, ACM, 2016, pp. 3–17.
[11] A. Panda *et al.*, "Netbricks: Taking the V Out of NFV," *Proc. USENIX OSDI '16*, USENIX, 2016.
[12] N. Dragoni *et al.*, "Microservices: Yesterday, Today, and Tomorrow," *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
[13] R. Mijumb *et al.*, "Network Function Virtualization: State-of-the-Art and Research Challenges," *IEEE Commun. Surveys & Tutorials*, vol. 18, no. 1, 2016, pp. 236–62.
[14] M. Kablan *et al.*, "Stateless Network Functions: Breaking the Tight Coupling of State and Processing," *Proc. USENIX NSDI*, 2017, pp. 97–112.
[15] A. Madhavapeddy *et al.*, "Unikernels: Library Operating Systems for the Cloud," *ACM SIGPLAN Notices*, vol. 48, no. 4. ACM, 2013, pp. 461–72.

Biographies

Shihabur Rahman Chowdhury is a Ph.D. student at the David R. Cheriton School of Computer Science, University of Waterloo. He received his B.Sc. degree in computer science and engineering from Bangladesh University of Engineering and Technology (BUET). His research interests include virtualization and softwarization of computer networks. He has received several scholarships and awards including co-recipient of the Best Paper award at IEEE/ACM/IFIP CNSM 2017.

Mohammad A. Salahuddin is a postdoctoral fellow at the David R. Cheriton School of Computer Science, University of Waterloo. He received his Ph.D. in computer science from Western Michigan University in 2014. His current research interests include the Internet of Things, content delivery networks, network softwarization, cloud computing, and cognitive network management. He serves as a TPC member for international conferences and is a reviewer for various journals and magazines.

Noura Limam received her M.Sc. and Ph.D. degrees in computer science from the University Pierre & Marie Curie, Paris VI, in 2002 and 2007, respectively. She is currently a research assistant professor of computer science at the University of Waterloo. She is on the Technical Program Committees and Organization Committees of several IEEE conferences. Her contributions are in the area of network and service management. Her current research interests are in network softwarization and cognitive network management.

Raouf Boutaba received his M.Sc. and Ph.D. degrees in computer science from the University Pierre & Marie Curie, Paris, in 1990 and 1994, respectively. He is a professor in the Cheriton School of Computer Science and Associate Dean, Research of the Faculty of Mathematics at the University of Waterloo, and holds an INRIA International Chair at INRIA Nancy. His research interests include network and service management, cloud computing, network virtualization, and network softwarization.